

Evaluating Large Language Models Trained on Code (Codex)

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. Petroski Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. Hebgén Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, W. Zaremba

arXiv (2021)

A language model for code

Program synthesis with language models

- Program synthesis is a longstanding challenge (Simon, 1963; Manna et al., 1971).
- Large code corpora became available, e.g. **CodeSearchNet** and **The Pile**.
- Self-supervised objectives used in NLP were adapted to code: **CodeBERT**, **T5**, and **PyMT5** are representative examples.
- Early analysis suggested that **GPT-3** could generate programs from Python docstrings even without explicit code-specialized training.
- Hypothesis of the paper: specializing a GPT language model for code could work across many tasks.

This work: Codex

Codex is a GPT-style model specialized for code. The paper positions it as a model that can generate useful code across tasks and notes that it is used for **Copilot**.

References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021); H. Husain et al., "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search, arXiv (2019); L. Gao et al., "The Pile: An 800GB Dataset of Diverse Text for Language Modeling, arXiv (2020); Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages, Findings of EMNLP (2020); T. Brown et al., "Language Models Are Few-Shot Learners, NeurIPS (2020); B. Wang and A. Komatsuzaki, "GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model, (2021).

Related Work

Learning to Execute

LSTMs are trained on programs and can learn algorithmic tasks such as adding two 9-digit numbers with about 99% accuracy. Curriculum learning plays a key role.

SPoC

The SPoC dataset contains about 18K C++ programs and studies pseudocode-to-code synthesis using a seq2seq model with best-first search.

Naturalness of Software

This work studies code with n-gram language models and shows that source code is statistically natural in ways that can be exploited for modeling and tooling.

APPS

APPS introduces a benchmark with 10K programming problems across three difficulty levels and includes test cases for evaluating generated solutions.

References: W. Zaremba and I. Sutskever, "Learning to Execute, arXiv (2014); A. Hindle et al., "On the Naturalness of Software, ICSE (2012); S. Kulal et al., "SPoC: Search-Based Pseudocode to Code, NeurIPS (2019); D. Hendrycks et al., "Measuring Coding Challenge Competence With APPS, NeurIPS (2021).

Importance: why this paper changed the field

Before Codex	What Codex changed	After Codex
• Program synthesis and code modeling already existed, but results were often shown on constrained tasks or with weaker text-similarity metrics. • The field still lacked a simple execution-based story for answering: can a language model actually solve a programming task?	• It combined a large code-specialized LM with a clean hand-written benchmark. • It made functional correctness and pass@k the central evaluation language for code generation. • It shifted the question from “can LMs autocomplete code?” to “how often can they solve a task?”	• The area moved toward stronger coding assistants, open code LMs, and harder benchmarks. • Follow-up work pushed on competition-level generation, open model releases, and repository-level software engineering evaluation.

References: M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv (2021); Y. Li et al., “Competition-Level Code Generation with AlphaCode,” Science (2022); L. Ben Allal et al., “StarCoder: May the Source Be with You!,” arXiv (2023); B. Roziere et al., “Code Llama: Open Foundation Models for Code,” arXiv (2023); J. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” ICLR (2024).

Task and approach

Task: functions from docstrings

- Input: **docstring**
- Output: **Python function**
- Code correctness is evaluated automatically via **unit tests**.
- This differs from natural language generation, which usually needs human assessors or heuristic metrics.
- For benchmarking, the paper constructs **164 original programming problems** with tests.
- Problem types include language comprehension, algorithms, simple mathematics, and interview-style questions.

Problem solving with one sample

Approach: generate samples and check whether any pass the unit tests.

- Codex (12B): **28.8%**
- Codex (300M): **13.2%**
- GPT-J (6B): **11.4%**
- Other GPT models: approximately **0%**
- After supervised fine-tuning on correct implementations, Codex-S (12B) reaches **37.7%**

Problem solving with multiple samples

Programming usually involves iteration and bug fixing. Repeated sampling approximates this process.

- Codex-S (12B) with 100 samples solves **77.5%** of problems.
- Selecting the sample with highest mean log-probability solves **44.5%**.

References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021); B. Wang and A. Komatsuzaki, "GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model, (2021).

Evaluation framework

Functional correctness

- Match-based metrics compare against a reference exactly or fuzzily (e.g. BLEU).
- Such metrics are limited for code because many different programs can be equivalent.
- The paper therefore emphasizes **functional correctness**: a sample is correct if it passes a set of unit tests.
- This is also how human developers judge code in practice, e.g. via test-driven development and CI.

The pass@k metric

Generate k samples per problem. A problem is solved if *any* sample passes the tests.

Unbiased estimator used in the paper:

$$\text{pass@}k \triangleq \mathbb{E}_{\text{problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

where $n \geq k$ is the number of samples and c is the number of correct samples among them.

Advantage: using more generated samples reduces variance while keeping comparisons fair.

References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021); K. Papineni et al., "BLEU: A Method for Automatic Evaluation of Machine Translation, ACL (2002); S. Ren et al., "CodeBLEU: A Method for Automatic Evaluation of Code Synthesis, arXiv (2020); B. Roziere et al., "Unsupervised Translation of Programming Languages, NeurIPS (2020); S. Kulal et al., "SPoC: Search-Based Pseudocode to Code, NeurIPS (2019).

Evaluation details

HumanEval: hand-written evaluation set

- Functional correctness is evaluated on **164 hand-written problems**.
- Each HumanEval problem contains a function signature, a docstring, a body, and several unit tests (average: **7.7** tests per problem).
- Hand-written problems matter because the models are trained on GitHub, where many public benchmark solutions already exist.
- HumanEval targets simple mathematics, reasoning, and language comprehension.
- The dataset is made public for future benchmarking.

Sandbox for executing generated programs

- Executing public code and generated code creates a security risk.
- The paper therefore runs untrusted programs in a sandboxed environment.
- **gVisor** is used to isolate the host, and network-adjacent services are protected with eBPF-based firewall rules.

References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021); D. Hendrycks et al., "Measuring Coding Challenge Competence With APPS, NeurIPS (2021); OpenAI, "HumanEval benchmark repository; Google, "gVisor sandboxed container runtime.

Code Fine-tuning

Overview

Codex is produced by fine-tuning GPT models of up to **12B** parameters. Unlike GPT models trained only on text, Codex obtains non-trivial HumanEval performance, and with 100 samples per problem, the majority of problems have at least one solution.

Data collection and fine-tuning

- Training corpus collected in May 2020 from **54M GitHub repositories**.
- Produced **179 GB** of unique Python files under 1 MB.
- Filtering removed likely auto-generated files, very long lines, and files with a very small fraction of alphanumerics.
- Final result: **159 GB** of unique Python files.
- Surprisingly, fine-tuning from GPT-3 gave no improvement over training from scratch, although it converged faster and was used in practice.

Optimization and tokenization

- Same learning rate as the corresponding GPT model.
- Linear warmup for 175 steps; cosine decay afterwards.
- Training on **100B tokens** with Adam ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$) and weight decay 0.1.
- GPT-3 tokenization is reused, but extra whitespace tokens are added for code.
- This reduces the token count for code by roughly **30%**.

References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021); D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization, ICLR (2015).

Supervised Fine-tuning

Overview

GitHub data contains far more than clean “function from docstring” examples: it also includes classes, config files, scripts, and data files. This mismatch may limit HumanEval performance, so the paper builds a supervised training set of standalone function-completion tasks.

Source 1: competitive programming problems

- Problems, solutions, and function signatures were collected from interview-preparation and programming-contest websites.
- Problem descriptions are used as docstrings.
- Unit tests are created from examples in statements and by submitting incorrect solutions to reveal hidden cases.
- These sources provide self-contained tasks with strong algorithmic content and usually good test coverage.
- Roughly **10,000** problems are curated from these sources.

References: M. Chen et al., “Evaluating Large Language Models Trained on Code, arXiv (2021).

Supervised Fine-tuning

Source 2: problems from Continuous Integration

- Problems are also extracted from open-source repositories by tracing inputs and outputs during integration tests with `sys.setprofile`.
- CI config files reveal how to create environments, install dependencies, and run tests.
- Repositories using **Travis** or **Tox** are particularly useful; additional source is pulled from PyPI.
- Because the code is untrusted, all integration tests are executed in the sandbox.
- Only about **40,000** problems are collected from millions of functions, mainly because not all functions are easy to trace and some runtime objects cannot be restored outside the sandbox.

Filtering problems

- Automatically gathered prompts may be ambiguous or stateful.
- Codex-12B is used to generate 100 samples per problem.
- If all samples fail, the problem is discarded as too hard or ambiguous.
- Verification is rerun multiple times to filter out stateful problems.

References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021); Travis CI documentation; Tox documentation.

Supervised Fine-tuning

Methodology - training with prompts

Training examples are assembled in the same prompt format used at evaluation time.

```
def solution(lst):  
    """Given a non-empty list of integers, return the sum  
    of all odd elements that are in even positions.
```

Examples

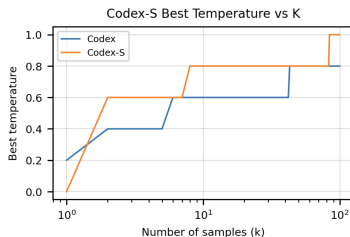
```
solution([5, 8, 7, 1]) => 12  
solution([3, 3, 3, 3, 3]) => 9  
solution([30, 13, 24, 321]) => 0
```

```
"""  
return sum(lst[i] for i in range(0, len(lst))  
          if i % 2 == 0 and lst[i] % 2 == 1)
```

- Codex is fine-tuned on these training problems to produce **Codex-S**.
- Training minimizes the negative log-likelihood of the reference solution while masking the prompt.
- Shorter prompts are left-padded so the target solutions line up.
- Learning rate is 1/10 of Codex with the same schedule until validation loss plateaus (after fewer than 10B tokens).

Optimal temperatures

- Codex-S prefers higher temperatures than Codex for all cases with $k > 1$.
- The paper suggests this may reflect a narrower learned distribution.
- For later evaluation:
 - $T^* = 0$ for pass@1
 - $T^* = 1$ for pass@100



References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021).

Comparative Analysis of Related Models

Related approaches

- GPT-Neo and GPT-J-6B are the closest comparison points.
- Both are trained on The Pile, of which about 8% comes from GitHub.
- GPT-J-6B already produces qualitatively reasonable code.

Temperatures

GPT-Neo: 0.2, 0.4, 0.8
GPT-J-6B: 0.2, 0.8
Tabnine: 0.4, 0.8

HumanEval results

Model	$k = 1$	$k = 10$	$k = 100$
GPT-Neo 125M	0.75%	1.88%	2.97%
GPT-Neo 1.3B	4.79%	7.47%	16.30%
GPT-Neo 2.7B	6.41%	11.27%	21.37%
GPT-J 6B	11.62%	15.74%	27.74%
Tabnine	2.58%	4.35%	7.59%
Codex-12M	2.00%	3.62%	8.58%
Codex-25M	3.21%	7.10%	12.89%
Codex-42M	5.06%	8.80%	15.55%
Codex-85M	8.22%	12.81%	22.40%
Codex-300M	13.17%	20.37%	36.27%
Codex-679M	16.22%	25.70%	40.95%
Codex-2.5B	21.36%	35.42%	59.50%
Codex-12B	28.81%	46.81%	72.31%

Takeaway

Codex-12B goes considerably beyond the performance of prior models.

Parameter note

Codex-300M achieves this despite having roughly **20x fewer parameters than GPT-J-6B**.

References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021); S. Black et al., "GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow, (2021); B. Wang and A. Komatsuzaki, "GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model, (2021); L. Gao et al., "The Pile: An 800GB Dataset of Diverse Text for Language Modeling, arXiv (2020).

Results on the APPS Dataset

APPS dataset comparison

- APPS was introduced to measure coding challenge competence.
- It contains **5000 train** and **5000 test** coding problems.
- Each example includes unit tests; training examples also include solutions.
- Most APPS tasks are **full-program synthesis** problems (stdin/stdout), not single-function generation.
- Original APPS metrics: **strict accuracy** and **test case average**.
- Codex results here are reported only under strict accuracy / pass@k.

Implementation details and results

- Example cases: APPS gives 3 I/O examples; Codex uses one example as a formatting hint ("1-shot").
- Timeouts: results are reported both with and without counting solutions that pass tests but time out after 3 seconds.

Method	Intro	Interview	Competition
GPT-Neo 2.7B raw p@1	3.90	0.57	0.00
GPT-Neo 2.7B raw p@5	5.50	0.80	0.00
1-shot Codex raw p@1	4.14 (4.33)	0.14 (0.30)	0.02 (0.03)
1-shot Codex raw p@5	9.65 (10.05)	0.51 (1.02)	0.09 (0.16)
1-shot Codex raw p@100	20.20 (21.57)	2.04 (3.99)	1.05 (1.73)
1-shot Codex raw p@1000	25.02 (27.77)	3.70 (7.94)	2.33 (5.85)
1-shot Codex filtered p@1	22.78 (25.10)	2.64 (5.78)	3.04 (5.25)
1-shot Codex filtered p@5	24.52 (27.15)	3.23 (7.13)	3.08 (5.53)

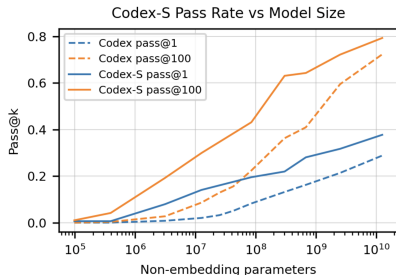
Temperature 0.6 is used for all k in pass@k.

References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021); D. Hendrycks et al., "Measuring Coding Challenge Competence With APPS, NeurIPS (2021); S. Black et al., "GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow, (2021).

Supervised Fine-tuning: Results

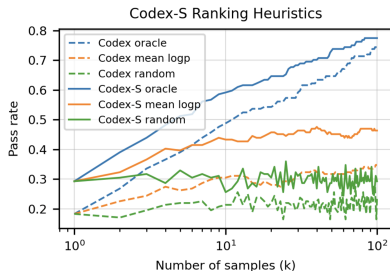
Influence of model size

- Codex-S beats Codex by an average of **6.5%** on pass@1.
- On pass@100, the average gain is **15.1%**.



Sample ranking heuristics

- Oracle reranking remains an upper bound.
- Mean log-probability is again the best practical heuristic.
- The average benefit of mean log-probability is about **2% higher** for Codex-S than for Codex.

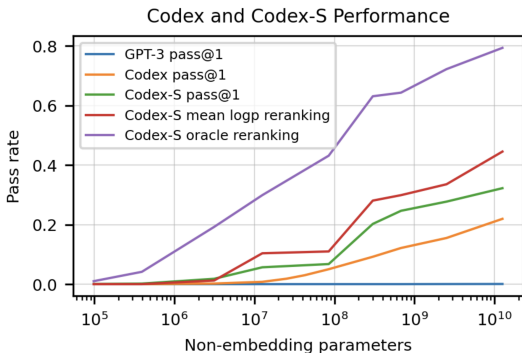


References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021).

Comparing Codex and Codex-S

Comparing training strategies on HumanEval

- **Codex-S mean-logp reranking** solves **44.5%** when reranking 100 samples.
- **Codex-S oracle reranking** solves **77.5%** when selecting from 100 samples using the unit tests.



Docstring complexity

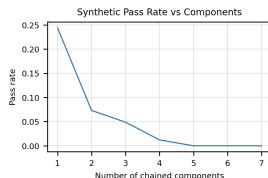
Composing building blocks

The 13 building blocks are chained by concatenating:

- their one-line descriptions into a docstring, and
- their one-line implementations into a code body.

```
def string_manipulation(s: str):  
    """  
    This function takes a string as input, then  
    returns the result of performing the following  
    sequence of manipulations:  
    - make every other character uppercase  
    - replace spaces with triple spaces  
    """  
    s = "".join(char.upper() if i % 2 == 0 else char  
                 for i, char in enumerate(s))  
    s = s.replace(" ", "  ")  
    return s
```

Results



- As each component is added, the pass rate drops by about **2–3x**.
- Human programmers do not show the same degradation once they can chain two operations.
- Codex also makes variable-binding mistakes when many variables are mentioned.

Example binding failure

Prompt idea: "Add 3 to y, then subtract 4 from both x and w. Return the product of the four numbers."

Observed failure: the generated code subtracts 4 from x but forgets to subtract 4 from w, and only multiplies two numbers.

Limitations: where the method breaks

Benchmark and evaluation limits

- **HumanEval is small and narrow:** only 164 problems, Python-only, and mostly single-function synthesis.
- **pass@k can look stronger than real one-shot use:** many-sample success is not the same as a reliable first answer.
- The benchmark omits repository context, debugging loops, retrieval, code review, and long-horizon maintenance.

Model failure modes

- Performance degrades on longer and more compositional docstrings.
- Multi-function prompts and more realistic full-program settings are harder than HumanEval.
- Generated code can be plausible but still wrong, insecure, or misaligned with user intent.

Takeaway

Codex is best viewed as a strong candidate generator plus verification story, not as a fully reliable end-to-end software engineer.

References: M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv (2021); D. Hendrycks et al., "Measuring Coding Challenge Competence With APPS," NeurIPS (2021).

Broader Impacts and Hazard Analysis

Applications of Codex

- onboarding users to new codebases,
- reducing context switches for experienced coders,
- enabling non-programmers to write specifications,
- producing draft implementations,
- aiding education and exploratory coding.

Hazard analysis and over-reliance

- Main risks: generated code may not match user intent, and the system may be misused.
- The analysis is framed in terms of risk factors rather than full product safety design.
- Over-reliance is a central concern, especially for novice programmers.
- Generated code may look correct but still be wrong or insecure.
- Automation bias makes this especially dangerous: people often trust automated suggestions too much.
- Human oversight remains necessary.

References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021); N. Leveson, "Improving the Standard Risk Matrix: Part 1, (2019).

What happened after Codex?

Three directions the line of work took next

1. **Harder generation and filtering**: systems such as AlphaCode pushed large-scale sampling and selection on competition-style tasks.
2. **Open code LMs**: models such as StarCoder and Code Llama broadened access, reproducibility, and multilingual code modeling.
3. **More realistic ML4SE evaluation**: benchmarks such as SWE-bench shifted attention from isolated functions toward repository-level issue resolution.

Codex did not solve software engineering. It changed the field by giving a compelling first demonstration, a reusable benchmark story, and a new default question for code LMs: not “can it write code?” but “can it solve tasks we can execute and verify?”

References: Y. Li et al., “Competition-Level Code Generation with AlphaCode,” *Science* (2022); L. Ben Allal et al., “StarCoder: May the Source Be with You!,” *arXiv* (2023); B. Roziere et al., “Code Llama: Open Foundation Models for Code,” *arXiv* (2023); J. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” *ICLR* (2024).

Summary

Summary

- The paper showed that a code-specialized language model can solve a non-trivial fraction of docstring-to-function tasks.
- Its deeper contribution was methodological: code generation should be judged by **functional correctness**, not just by textual similarity.
- The strongest results rely on sampling, reranking, and task-aligned fine-tuning, so benchmark success should not be confused with reliable one-shot software engineering.
- Historically, Codex helped launch the modern code-assistant era and influenced the move toward stronger benchmarks and more realistic ML4SE settings.

References: M. Chen et al., "Evaluating Large Language Models Trained on Code, arXiv (2021).

Q&A